

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-10 Créditos: 3	
---	--	---

Taller 4: Planificación Automatizada

Tomas camilo Ostos Medina –202123214

Camila Fajardo – 202220711

Juan Francisco Rodríguez - 202214603

Luis Hernan Bobadilla Moreno - 202223854

Punto 1

a) Representacion del dominio

Para modelar el problema de rescate se utilizó una representación inspirada en PDDL, donde el dominio se describe mediante predicados, acciones, precondiciones y efectos. El estado del mundo se representa como un conjunto de flujos lógicos que describen la posición del robot, la ubicación de los pacientes, los suministros y los puestos médicos.

Las acciones implementadas fueron Move, PickUp, PutDown, Rescue y SetupSupplies. Cada acción contiene precondiciones positivas y negativas que determinan cuándo puede ejecutarse, así como efectos positivos y negativos que modifican el estado. Esto permitió representar correctamente la dinámica del entorno y garantizar que las transiciones entre estados fueran válidas.

b) Aplicabilidad de acciones

Se implementó un mecanismo de verificación de aplicabilidad para determinar si una acción puede ejecutarse en un estado dado. Una acción se considera aplicable únicamente si todas sus precondiciones positivas están presentes en el estado actual y ninguna de sus precondiciones negativas se encuentra activa.

Este proceso evita generar estados inconsistentes y garantiza que únicamente se expandan estados válidos durante la búsqueda.

c) Actualización de estados

La actualización de estados se realizó utilizando el modelo clásico STRIPS. Cuando una acción es aplicada, primero se eliminan del estado los flujos contenidos en la delete list, y posteriormente se agregan los flujos de la add list.

Esto permite modelar correctamente cambios como mover el robot, recoger objetos o rescatar pacientes.

d) Verificación experimental

La implementación fue probada en distintos layouts del entorno de rescate, verificando que las acciones solo fueran aplicables cuando correspondía, los estados se actualizaran correctamente y las metas pudieran alcanzarse mediante secuencias válidas de acciones.

Los resultados obtenidos mostraron que el dominio fue modelado correctamente y permitió ejecutar exitosamente los algoritmos de planificación implementados posteriormente.

Punto 2

a) Comportamiento BFS

En los resultados se observa que ForwardBFS expande más estados cuando el layout es más complejo o cuando el plan es más largo. Por ejemplo, en tinyhouse solo expandió 70 nodos y encontró un plan de longitud 9, mientras que en warehouseR expandió 463 nodos y encontró un plan de longitud 31. Esto muestra que, al aumentar la distancia y las acciones necesarias para completar el rescate, también aumenta el número de estados explorados.

Comparando cornerRescue y mediumRescue, cornerRescue tuvo 118 nodos expandidos y un plan de longitud 19, mientras que mediumRescue tuvo 326 nodos y un plan de longitud 32. En general, BFS se comportó de forma estable y encontró planes válidos, pero su costo crece con la profundidad del plan y el tamaño del espacio de búsqueda.

ForwardBFS

Layout	Nodos expandidos	Longitud del plan	Tiempo (sec)
cornerRescue	118	19	0.156
mediumRescue	326	32	0.437

narrowRescue	116	23	0.110
openRescue	371	26	0.648
smallRescue	131	23	0.142
warehouseR	463	31	0.994
tinyhouse	70	9	0.026

b) Complejidad espacial: $O(b^d \cdot (n + d))$

b representa el factor de ramificación efectivo después de aplicar las podas, d es la profundidad del plan y n es el número de hechos lógicos del estado. Esto se debe a que la búsqueda guarda en memoria la frontera y el conjunto visited para evitar repetir estados. Cada estado está representado como un conjunto de hechos lógicos, por lo que puede ocupar hasta $O(n)$. Además, en la frontera también se almacena el plan parcial asociado a cada estado, que puede ocupar hasta $O(d)$. Aunque las podas reducen la cantidad real de estados almacenados, en el peor caso la memoria sigue creciendo exponencialmente con la profundidad del plan.

c) Complejidad temporal: $O(b^d \cdot a \cdot n)$

El algoritmo puede expandir hasta $O(b^d)$ estados en el peor caso, y para cada estado llama a `getSuccessors`, que genera las acciones aplicables. Además, se revisa si cada acción es útil para el rescate y si el nuevo estado ya fue visitado. Como estas verificaciones pueden implicar recorrer conjuntos de hechos lógicos, el costo puede depender de n. En la práctica, las podas reducen el número de acciones exploradas, pero no eliminan el crecimiento exponencial del peor caso.

d) Completitud:

El algoritmo es completo bajo la condición de que las podas no eliminen acciones necesarias para llegar a una solución válida. Una BFS pura sí es completa en espacios finitos porque explora los estados por niveles y usa visited para evitar ciclos. Sin embargo, esta implementación filtra algunas acciones mediante `is_useful_for_rescue`, por ejemplo evitando soltar suministros o pacientes en lugares que no aportan directamente al objetivo. En los layouts probados estas podas ayudan a reducir caminos inútiles, pero teóricamente la completitud depende de que esas reglas no descarten una acción que sí sea necesaria en algún caso particular.

e) Optimalidad:

La optimalidad tampoco está garantizada de forma absoluta. Una BFS pura con acciones de costo uniforme sí encuentra el plan más corto en número de acciones, porque explora primero los planes de menor profundidad. En esta implementación, como se aplican podas específicas del dominio, la optimalidad depende de que no se elimine ningún camino válido que pudiera llevar a un plan más corto. En los experimentos realizados el algoritmo encontró planes válidos y eficientes, pero desde el punto de vista teórico solo se puede afirmar que sería óptimo si todas las acciones tienen el mismo costo y las podas son seguras.

f) Uso de IAG:

Durante el desarrollo utilicé IA generativa como apoyo para revisar errores, mejorar la explicación del algoritmo y analizar su complejidad. La IA me ayudó a entender que esta implementación no era una BFS pura, sino una búsqueda hacia adelante con podas específicas del dominio de rescate. También me permitió identificar cómo esas podas afectan la completitud y la optimalidad del algoritmo. Sin embargo, la validación final dependió de ejecutar el código, revisar los resultados obtenidos en los layouts y comprobar que los planes generados fueran correctos.

Punto 3

a) búsqueda regresiva vs. búsqueda hacia adelante

En los layouts evaluados, ForwardBFS fue más eficiente que BackwardSearch en todos los casos. Aunque ambos algoritmos encontraron planes con la misma longitud, ForwardBFS expandió muchos menos nodos y tuvo tiempos de ejecución mucho menores. Por ejemplo, en narrow, ForwardBFS expandió 66 nodos en 0.029 segundos, mientras que BackwardSearch expandió 236122 objetivos parciales en 40.298 segundos.

Esto muestra que, en este dominio, la búsqueda regresiva fue más costosa porque genera muchas combinaciones de objetivos parciales, especialmente relacionadas con las posiciones del robot, los suministros y el paciente. En cambio, ForwardBFS solo expande acciones aplicables desde el estado actual, lo que resultó más controlado en estos layouts.

La búsqueda regresiva sería más eficiente en problemas donde el objetivo sea muy específico y pocas acciones puedan lograrlo directamente. Sin embargo, en problemas con muchos movimientos, varias rutas posibles y muchas condiciones intermedias, como este caso de rescate, puede explorar demasiados objetivos parciales y volverse menos eficiente.

ForwardBFS

Layout	Nodos expandidos	Longitud del plan	Tiempo (sec)
narrow	66	16	0.029
farSupplies	51	11	0.011
hallway	44	15	0.024
openSmall	42	13	0.013
tinyhouse	70	9	0.026

BackwardSearch

Layout	Nodos expandidos	Longitud del plan	Tiempo (sec)
narrow	236122	16	40.298
farSupplies	41891	11	10.203
hallway	16592	15	1.681
openSmall	82264	13	8.238
tinyhouse	14479	9	2.690

b) Complejidad espacial: $O(a^d * n)$

Donde a representa el número de acciones aterrizadas, d la profundidad del plan y n el número de hechos lógicos del estado. El algoritmo puede almacenar muchos objetivos parciales en la frontera y en el conjunto de visitados. En el peor caso, se pueden generar hasta $O(a^d)$ objetivos parciales. Cada objetivo parcial puede ocupar hasta $O(n)$, ya que está compuesto por un conjunto de hechos lógicos. Además, como en la frontera también se almacena el plan parcial asociado a cada objetivo, este puede ocupar hasta $O(d)$.

c) Complejidad temporal: $O(a^d * n * a)$

El algoritmo puede expandir hasta $O(a^d)$ objetivos parciales en el peor caso. Para cada objetivo parcial, puede llegar a revisar hasta $O(a)$ acciones aterrizadas para verificar si son relevantes y aplicar la regresión. Además, cada operación de regresión y cada verificación de consistencia compara conjuntos de hechos lógicos, lo que puede costar hasta $O(n)$. Aunque el algoritmo incluye filtros para descartar acciones inválidas y objetivos inconsistentes, estas mejoras reducen el tiempo en la práctica, pero no cambian el peor caso teórico.

d) Completitud:

Es completo si el espacio de búsqueda es finito y las podas no eliminan soluciones válidas. En este caso, como hay una cantidad limitada de objetos, también hay

una cantidad limitada de acciones y objetivos parciales. Además, se usa “visited” para no repetir objetivos ya explorados. Las podas solo buscan quitar casos claramente imposibles, como que un objeto esté en dos lugares al mismo tiempo o que el robot tenga las manos libres mientras sostiene algo.

e) Optimalidad:

El algoritmo no garantiza optimalidad en todos los casos. Aunque la búsqueda avanza por niveles usando una frontera, también aplica filtros y ordena las acciones para explorar primero algunas antes que otras. Esto puede mejorar el tiempo de ejecución, pero no asegura formalmente que el primer plan encontrado sea siempre el de menor costo. La optimalidad solo estaría garantizada si todas las acciones tuvieran el mismo costo, la búsqueda explorara estrictamente por profundidad como BFS y las podas no eliminaran ningún camino válido que pudiera llevar a un plan más corto. En este caso, el algoritmo busca encontrar un plan válido de manera más eficiente, pero no se puede asegurar que siempre sea el plan óptimo.

f) Uso IAG

Durante el desarrollo del taller utilizamos la IA como apoyo para revisar errores y mejorar la implementación. Al inicio fue difícil formular prompts precisos, porque no bastaba con preguntar de forma general. Fue más útil compartir el código, los errores de terminal y los resultados de ejecución. Las correcciones me ayudaron a entender mejor la diferencia entre una solución que funciona teóricamente y una que también es eficiente en la práctica, ya que algunos enfoques resultaban demasiado costosos computacionalmente. La IA fue útil como apoyo para depurar y orientar mejoras, pero la validación final dependió de probar el código y analizar sus resultados. También fue útil para mejorar los layouts que se usaron para poder experimentar y validar con el algoritmo.

Punto 4a

a) Se implementa la heurística ignorePreconditionsHeuristic eliminando todas las precondiciones de las acciones del dominio. Al estar relajada, cualquier acción puede ejecutarse en cualquier estado.

La heurística calcula cuántas acciones son necesarias para satisfacer los flujos del objetivo que todavía no están presentes en el estado actual. Para esto utilizamos una estrategia greedy tipo set-cover:

- Se identifican los flujos del objetivo que aún no están satisfechos.

- Se revisan las acciones del dominio y sus `add_list`.
- Se escoge la acción que cubra la mayor cantidad de objetivos faltantes.
- Se eliminan esos objetivos de la lista pendiente.
- El proceso se repite hasta cubrir todos los fluentes posibles.

El número de acciones seleccionadas corresponde al valor heurístico.

b) Demostración de admisibilidad

La heurística es admisible porque nunca sobreestima el costo real del problema. Se utilizó el layout `tinyBase` para verificar este comportamiento. En el problema real, para rescatar al paciente se necesita ejecutar múltiples acciones:

- Moverse hacia los suministros.
- Recoger los suministros.
- Llevar los suministros al puesto médico.
- Preparar los suministros.
- Moverse hacia el paciente.
- Recoger al paciente.
- Llevar al paciente al puesto médico.
- Soltar al paciente.
- Ejecutar la acción `Rescue`.

El costo real del plan encontrado fueron 9 acciones. Sin embargo, la heurística ignora completamente las precondiciones. Debido a esto, la acción: `Rescue(robot, patient_0, (1,2))` puede considerarse aplicable, ya que únicamente se analiza su `add_list`. Con esto, la heurística estima un costo mucho menor que el costo real. Como el valor heurístico nunca supera el costo verdadero de la solución, la heurística es admisible.

Punto 4b

a) La heurística `ignoreDeleteListsHeuristic` fue implementada eliminando todos los efectos negativos (`delete list`) de las acciones. En este problema relajado, los fluentes nunca desaparecen del estado, por lo que el estado solo puede crecer y nunca se pierde progreso. La implementación utiliza `hill-climbing greedy`:

- Se parte del estado actual.
- Se generan las acciones aplicables.
- En cada iteración se selecciona la acción que agregue la mayor cantidad de fluentes del objetivo aún no satisfechos.
- Los nuevos fluentes se agregan al estado relajado sin eliminar ninguno.
- El proceso continúa hasta satisfacer el objetivo.

El número de pasos realizados corresponde al valor heurístico esperado.

b) Se implementó el algoritmo aStarPlanner utilizando búsqueda forward con prioridad $f(n) = g(n) + h(n)$ donde:

$g(n)$ corresponde al costo acumulado real del plan.

$h(n)$ corresponde al valor retornado por la heurística.

La frontera se gestiona mediante una PriorityQueue, expandiendo primero los estados con menor valor de $f(n)$. Además, se almacena el mejor costo conocido para cada estado para evitar expansiones redundantes.

c) Se realizaron pruebas sobre el layout tinyBase usando las dos heurísticas implementadas.

Resultados

Metodo	Nodos expandidos	Tiempo	Longitud del plan
A* + ignorePreconditions	701	0.126 s	9
A* + ignoreDeleteLists	315	15.523 s	9

En ambos casos se encontró un plan válido de 9 acciones.

d) La heurística ignoreDeleteLists resultó ser más informativa en el dominio de rescate. Esto se puede observar porque el número de estados expandidos fue considerablemente menor ($315 < 701$). La razón es que esta heurística todavía respeta las precondiciones de las acciones y únicamente elimina los efectos negativos. Gracias a esto, la estimación obtenida es más cercana al costo real del problema. Por otro lado, ignorePreconditions produce estimaciones más débiles, ya que considera que cualquier acción puede ejecutarse inmediatamente. Aunque ignoreDeleteLists expandió menos estados, el tiempo total fue mayor debido al costo computacional añadido de calcular la heurística mediante hill-climbing sobre el problema relajado. Con esto podemos concluir que ignoreDeleteLists es más informativa, mientras que ignorePreconditions es más rápida pero menos precisa.

g) Uso de IAG:

Se utilizó la IA como apoyo para revisar y mejorar la implementación de búsqueda regresiva. Al inicio intentamos hacerla con un enfoque tipo BFS, pero era demasiado lenta y en algunos layouts la frontera crecía demasiado, por lo que no terminaba en un tiempo razonable. Con ayuda de la IA pudimos identificar que el problema estaba en la cantidad de objetivos parciales innecesarios que se estaban generando y a partir de eso ajustamos el enfoque hacia una búsqueda

más informada, usando prioridad con heurística y filtros para descartar acciones u objetivos inconsistentes. La IA no reemplazó mi trabajo, pero sí me ayudó a depurar errores, interpretar los resultados de ejecución y entender por qué ciertos cambios mejoraban el rendimiento.

Punto 5a

En `planning/htn.py` se implementaron las acciones de alto nivel del dominio de rescate. Cada HLA representa una tarea abstracta que se descompone en acciones más simples hasta llegar a acciones primitivas del dominio PDDL. Las HLAs implementadas fueron:

- **Navigate(from, to):**

Se refina en una secuencia de acciones primitivas Move, calculadas usando las relaciones Adjacent del estado inicial.

- **PrepareSupplies(s, m):**

Navigate hasta el suministro, Pickup, Navigate hasta el puesto médico y SetupSupplies.

- **ExtractPatient(p, m):**

Navigate hasta el paciente, Pickup, Navigate hasta el puesto médico y PutDown.

- **FullRescueMission(s, p, m):**

PrepareSupplies, luego ExtractPatient y finalmente Rescue.

También se implementó **FullRescueProgram** como HLA raíz. Esta HLA genera la misión completa que debe resolver el problema. Para el caso simple contiene una misión de rescate; para el caso multi-rescate permite generar una secuencia de misiones para varios pacientes.

El algoritmo **hierarchicalSearch** realiza una búsqueda en amplitud sobre planes jerárquicos. Comienza con la HLA raíz, encuentra la primera HLA no primitiva del plan y la reemplaza por uno de sus refinamientos. Cuando el plan ya solo contiene acciones primitivas, se simula desde el estado inicial usando `is_applicable` y `apply_action`. Si el estado final cumple el objetivo, el plan se retorna como solución.

Resultados:

Layout	Nodos expandidos	Longitud del plan	Tiempo (sec)
tinyHTN	10	9	0.000s
htnBase	10	12	0.000s

Punto 5b

La implementación HTN se extendió para manejar MultiRescueProblem. Para resolverlo, se creó una HLA raíz llamada FullRescueProgram, que genera una secuencia de tareas FullRescueMission, una por cada paciente. Cada FullRescueMission se encarga de preparar suministros si el puesto médico aún no está listo, extraer al paciente y ejecutar la acción final Rescue.

La implementación también considera distintos órdenes posibles de pacientes usando permutaciones. Esto es importante porque el costo total del plan depende de la posición inicial del robot, la ubicación de los pacientes, los suministros y el puesto médico. Si se rescata primero un paciente cercano, el camino posterior puede ser más corto o más largo dependiendo de dónde queda el robot después de esa misión.

Resultados:

Layout	Nodos expandidos	Longitud del plan	Tiempo (sec)
duoRescue	252	20	0.015s
crossMulti	96	33	0.007s

Complejidad de HTN:

En el peor caso, hierarchicalSearch explora un árbol de refinamientos con factor de ramificación b y profundidad d . Con longitud del plan primitivo final L y número de pacientes en MultiRescueProblem m .

Aspecto	Complejidad
Tiempo	$O(b^d * L)$
Espacio	$O(b^d * L)$

Punto 6a

Los resultados experimentales obtenidos muestran diferencias importantes entre los algoritmos implementados. En el caso de ForwardBFS, el algoritmo logró encontrar soluciones válidas en todos los layouts evaluados, manteniendo tiempos de ejecución relativamente bajos en mapas pequeños y medianos. Sin embargo, a medida que aumentó el tamaño del entorno y el número de acciones posibles, también aumentó significativamente la cantidad de estados expandidos.

Por otro lado, la búsqueda regresiva mostró comportamientos distintos dependiendo de la estructura del problema. En layouts simples y con metas específicas, como tinyhouse, backwardSearch logró explorar menos nodos y encontrar soluciones más rápidamente. No obstante, en problemas más grandes el algoritmo generó una gran cantidad de objetivos parciales, causando un crecimiento acelerado de la frontera y tiempos de ejecución mucho mayores.

Estos resultados evidencian cómo la estructura del espacio de búsqueda afecta directamente el desempeño de los algoritmos de planificación.

Punto 6b

ForwardBFS presenta como principal ventaja su completitud y su capacidad de encontrar soluciones óptimas en términos de longitud del plan cuando todas las acciones tienen el mismo costo. Además, su comportamiento es más estable en distintos layouts.

Sin embargo, su principal limitación es el crecimiento exponencial del espacio de estados, lo cual genera altos costos computacionales en problemas grandes.

En contraste, backwardSearch puede ser más eficiente cuando las metas son pequeñas y existen pocas acciones relevantes para alcanzarlas. Aun así, su desempeño se deteriora considerablemente en dominios donde existen múltiples caminos, acciones de movimiento o muchos subobjetivos intermedios.

Respecto a las heurísticas implementadas, se observó que ignoreDeleteLists genera estimaciones más informativas y reduce el número de estados expandidos, aunque con un costo computacional mayor. Por el contrario, ignorePreconditions produce estimaciones más rápidas pero menos precisas.

Punto 6c

La inteligencia artificial generativa fue utilizada como una herramienta de apoyo durante el desarrollo del taller, principalmente para validar ideas de implementación, depurar errores y comprender mejor ciertos comportamientos de los algoritmos de planificación.

Inicialmente, algunos enfoques implementados presentaban problemas de rendimiento, especialmente en la búsqueda regresiva, donde el crecimiento de la frontera hacía que ciertos layouts fueran demasiado costosos computacionalmente. La IA ayudó a identificar posibles causas del problema y permitió explorar estrategias más informadas para controlar la generación de objetivos parciales.

Sin embargo, el diseño del dominio, la implementación de las acciones, la interpretación de los resultados y la validación experimental fueron realizados manualmente. En este sentido, la IA funcionó como una herramienta complementaria al proceso de aprendizaje y no como un reemplazo del trabajo realizado por el grupo.